a.k.a. "there is no global undo button"

Saga

TL;DR

When one business action (like "place an order") touches several services, each with its own database, split it into a **sequence of local transactions**. Each step commits in its own service. If a later step fails, run **compensating actions** that undo the earlier steps, in reverse order.

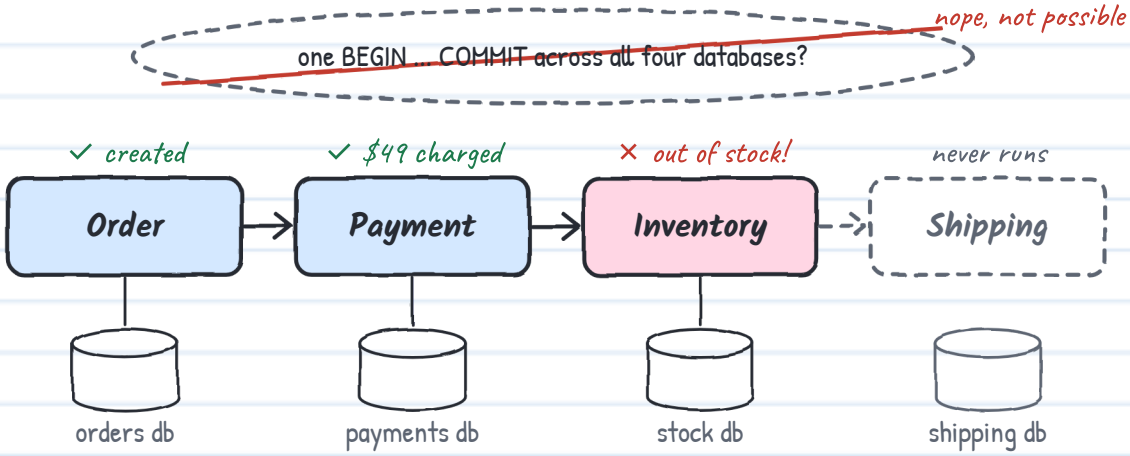
Family: *data management / consistency* | **Two styles:** *choreography & orchestration*

First described: *Garcia-Molina & Salem, 1987*

Pairs with: *outbox, idempotency, event-driven messaging*

1 The problem *why can't we just use a transaction?*

In a monolith, "place order" is one database transaction: `BEGIN`, do everything, `COMMIT`. If anything fails, `ROLLBACK` and it's as if nothing happened. In microservices each service owns its own database, so that safety net is gone.



the customer paid $49 for nothing, and nothing rolls it back

▲ *Each service committed its own part. When step 3 fails, steps 1 and 2 are already permanent.*

So the questions are: how do we keep the data **consistent** across services, and what do we do with the steps that already succeeded?

Why not two-phase commit (2PC)?

2PC (XA transactions) *can* make several databases commit together, but it's a poor fit for microservices:

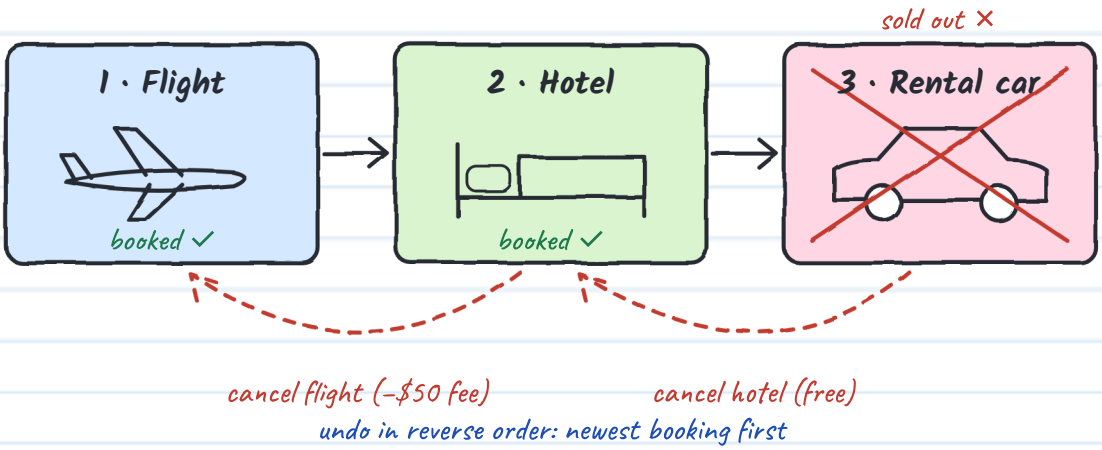
- Every participant **holds locks** until the coordinator says "commit". Slow participant = everyone waits.
- The coordinator is a **single point of failure**. If it dies mid-way, participants can be stuck "in doubt".
- Availability drops: the transaction succeeds only if *every* service is up at the same moment.
- Many modern stores and brokers (Kafka, most NoSQL databases, SaaS APIs) don't support it at all.

> ↳ *CAP reminder: across a network you pick availability OR strong consistency during a partition. Sagas pick availability + eventual consistency.*

2 Real-life analogy *booking a trip*

You're planning a holiday. You book a **flight** with an airline, a **hotel** on a booking site and a **rental car** from a car company. Three different companies, and there's no single button that says "book all three or none".

- Flight booked ✓, hotel booked ✓, then the car company says **sold out**.
- No car means the trip doesn't work. So you **cancel the hotel**, then **cancel the flight**, newest first.
- The flight cancellation costs a **$50 fee**. Cancelling isn't the same as it never happening. That's the big difference between a compensation and a database rollback.
- Smart travellers book the hard-to-get, non-refundable thing **last**, after everything else is confirmed.



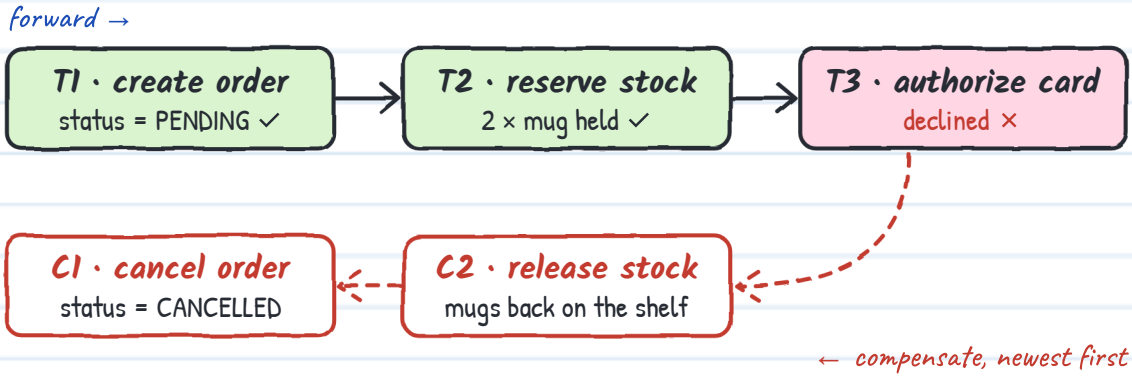
▲ *Each booking is its own "transaction". Cancelling is a new action with its own cost, not a rewind.*

Translating the trip to software

The trip	The saga
Each booking with a different company →	A **local transaction** in one service, committed on its own
Cancelling a booking →	A **compensating transaction** (a semantic undo)
The $50 cancellation fee →	Compensation isn't a perfect rollback; side effects can remain
A travel agent who books everything for you →	**Orchestration**: one coordinator tells each service what to do
A group chat where each friend books after the last one posts "done!" →	**Choreography**: services react to each other's events
Booking the non-refundable thing last →	The **pivot** step: once it succeeds, the saga only moves forward
Your hotel shows "pending" until the card clears →	A **semantic lock**: a PENDING status others can see

3 How it solves it *steps + undo steps*

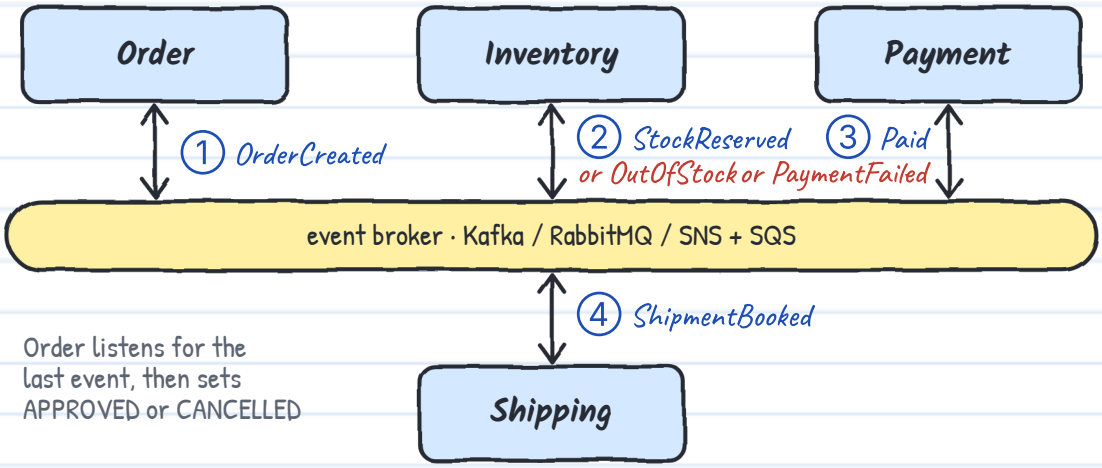
For each step T_i you design a matching compensation C_i. Run T_1, T_2, T_3… If T_3 fails, run C_2 then C_1. Every step commits right away in its own database, so **no locks are held across services.**



▲ *The order never "disappears". It ends as CANCELLED, which is a real, visible state.*

Style A · Choreography *(no boss, everyone listens)*

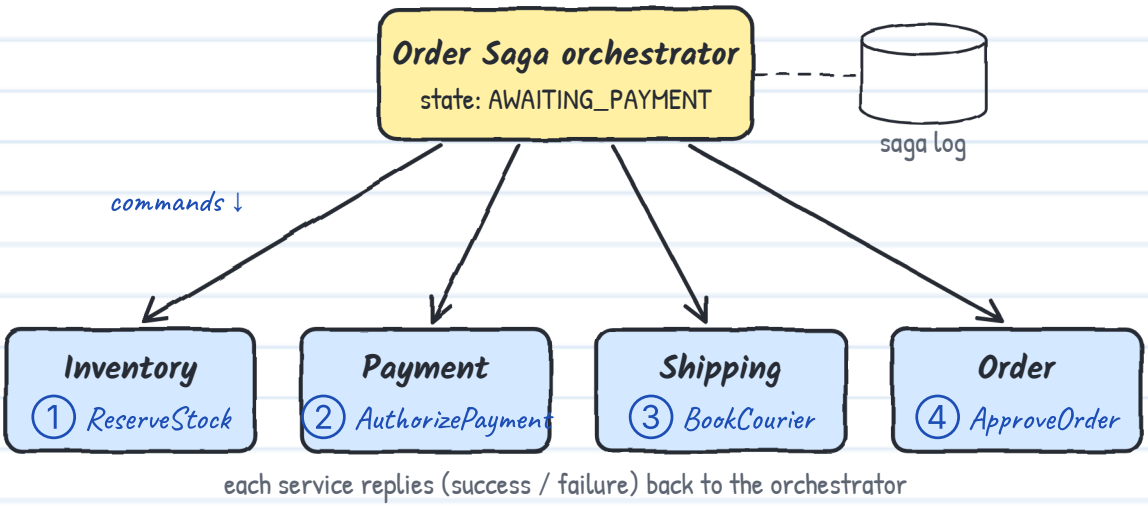
There's no central coordinator. Each service does its step, then **publishes an event**. The next service is subscribed to that event and reacts. Failures are events too (`PaymentFailed`), and the services that already acted listen for them and compensate.



▲ *Loose coupling and no single point of failure, but the full flow isn't written down anywhere.*

Style B · Orchestration *(one coordinator runs the show)*

A dedicated **saga orchestrator** knows the whole recipe. It sends a **command** to each service ("reserve stock for order 42"), waits for the reply, records progress in a **saga log**, and decides the next step or which compensations to run.



▲ *Easy to read, test and monitor in one place. The orchestrator must itself be durable and highly available.*

Pick choreography when...

The flow is short (2–4 steps), linear, and services are owned by different teams who want to stay loosely coupled.

Pick orchestration when...

The flow is long, has branches or timeouts, or the business wants to see "where is order 42 stuck?" in one place.

Designing the steps: our order saga

Order the steps so the ones most likely to fail and easiest to undo come first.

Step (T_i)	Compensation (C_i) / type
1. Order: create order with status **PENDING** →	Set status **CANCELLED** · compensatable
2. Inventory: reserve 2 × mug →	Release the reservation · compensatable
3. Payment: charge $49 →	**Pivot**: point of no return. If it succeeds, we only go forward
4. Shipping: book the courier →	Retry until it works · retriable
5. Order: set **APPROVED**, send confirmation email →	Retry until it works · retriable

> ↳ *rule of thumb from the lecture: compensatable steps → pivot → retriable steps. Never put a step you can't undo before one that might still fail.*

Vocabulary for the exam

Local transaction

An ACID transaction inside one service's own database.

Compensating transaction

A new action that semantically undoes an earlier step (refund, release, cancel).

Pivot transaction

The go/no-go step. After it commits, the saga must run to completion.

Retriable transaction

A step after the pivot that is guaranteed to succeed eventually if you keep retrying.

Semantic lock

A flag like PENDING that tells everyone else "this record is mid-saga".

Transactional outbox

Write the event to an outbox table in the same DB transaction, then publish it, so the update and the event can't drift apart.

run-saga.ts *orchestration, simplified*

```ts
type Step<C> = {
  name: string;
  run: (ctx: C) => Promise<void>;           // a local transaction in ONE service
  compensate?: (ctx: C) => Promise<void>;   // semantic undo, must be idempotent
  pivot?: true;                              // after this, never go back
};

export async function runSaga<C>(sagaId: string, steps: Step<C>[], ctx: C) {
  const done: Step<C>[] = [];
  let pastPivot = false;
  for (const step of steps) {
    try {
      await withRetry(() => step.run(ctx));            // idempotency key = sagaId + step
      await sagaLog.save(sagaId, step.name, "DONE");   // survive a crash, resume later
      done.push(step);
      if (step.pivot) pastPivot = true;
    } catch (err) {
      if (pastPivot) { await alertHuman(sagaId, step.name, err); throw err; } // can't undo now
      await sagaLog.save(sagaId, step.name, "COMPENSATING");
      for (const s of [...done].reverse()) {             // newest first
        if (s.compensate) await withRetry(() => s.compensate!(ctx));
      }
      throw new SagaAborted(sagaId, step.name, err);
    }
  }
}

await runSaga(order.id, [
  { name: "createOrder",  run: c => orders.create(c, "PENDING"), compensate: c => orders.cancel(c) },
  { name: "reserveStock", run: c => inventory.reserve(c),        compensate: c => inventory.release(c) },
  { name: "chargeCard",   run: c => payments.charge(c), pivot: true },
  { name: "bookCourier",  run: c => shipping.book(c) },
  { name: "approveOrder", run: c => orders.approve(c) },
], ctx);
```

↳ *real tools so we don't build the log/retry/resume ourselves:*
Temporal, AWS Step Functions, Azure Durable Functions, Camunda

> / Zeebe, MassTransit sagas (.NET), Axon / Eventuate Tram (Java).

4 Gains vs losses *nothing is free*

What we gain

+ **Consistency across services** without distributed locks or 2PC.
+ **Each service keeps its own database** and can be deployed and scaled on its own.
+ **High availability.** No coordinator holding locks; a slow service delays one saga, not everyone.
+ **Long-running processes** are fine: a step can wait minutes or days (e.g. manual approval).
+ **Failure handling is explicit** and written in business terms (refund, release, apologise).

What we lose

− **Only eventual consistency.** Others can see in-between states like PENDING.
− **No isolation** (the "I" in ACID). Concurrent sagas can cause lost updates or dirty reads unless you add countermeasures.
− **Much more code.** Every step needs a compensation, and both must be idempotent and retryable.
− **Some things can't be undone.** A sent email or shipped parcel needs a semantic fix, like an apology email or a return label.
− **Harder to debug.** One business action spans many logs; you need correlation IDs and tracing.

The "no isolation" gotcha and its countermeasures

- **Semantic lock:** mark records PENDING so other requests know to wait, or treat them carefully.
- **Commutative updates:** design changes so the order doesn't matter (add/subtract, not "set to").
- **Pessimistic ordering:** move risky steps earlier so dirty data is less likely to be read.
- **Re-read before acting:** check the value hasn't changed (version number) before overwriting it.

5 Architecture ratings *1 = poor · 5 = great*

Same nine characteristics as the architecture styles. Baseline here: a microservices system that tries to keep cross-service data consistent with **distributed 2PC**, compared with the same system using **sagas**.

Characteristic	With the pattern	Before → after	Why
Simplicity *easy to understand?*		3 → 2 ▼ −1	Every step needs an undo, and you must reason about in-between states.
Low cost *cheap to build & run?*		3 → 3 = no change	More code and a broker or workflow engine, but no XA-capable infrastructure.
Deployability *safe, easy releases?*		2 → 4 ▲ +2	Services stay independent. Changing the event or command contract still needs care.
Testability *test with confidence?*		3 → 2 ▼ −1	Many failure paths: each step failing, each compensation failing, duplicates, timeouts.
Scalability *more users & data?*		2 → 5 ▲ +3	No global locks, so each service scales on its own.
Elasticity *sudden spikes?*		2 → 4 ▲ +2	Async steps can queue up during a spike and drain afterwards.
Fault tolerance *failure stays contained?*		2 → 4 ▲ +2	No blocking coordinator. Failures become explicit, recoverable states.
Performance *fast typical request?*		2 → 4 ▲ +2	No lock waiting. The user gets "order received" fast, while the full result is eventual.
Evolvability *easy to change?*		2 → 4 ▲ +2	Add a step or service without touching the others (orchestration makes this easier).



6 Common pitfalls *copy these into the exam cheat-sheet*

✕ Compensations that aren't idempotent

The broker delivers "refund order 42" twice, and the customer is refunded twice.

Fix → idempotency keys (sagaId + step) and a "processed messages" table.

✕ The dual-write trap

The service updates its database, then crashes before publishing the event. The saga is stuck forever. ***Fix*** → transactional outbox (or change-data-capture) so the update and the event commit together.

✕ Exposing in-between states

The "Your order is confirmed!" email goes out at step 1, then payment fails at step 3. ***Fix*** → semantic lock (PENDING) and send confirmations only in a final, retriable step.

Thinking compensation = rollback

You can't un-send an email or un-ship a parcel. **Fix** → design semantic undos (apology email, return label) and put irreversible steps after the pivot.

No timeouts on steps

A service never replies and the saga waits forever, with stock reserved for nobody. **Fix** → a deadline per step, then compensate or escalate, and a dashboard of "stuck sagas".

Choreography spaghetti

Eight services reacting to each other's events; nobody can explain the full flow and events start going in circles. **Fix** → switch to orchestration once the flow passes about 4 steps or has branches.

A saga where a transaction would do

The data actually lives in one database, but the team builds a saga anyway. **Fix** → use a normal local ACID transaction. Sagas are for crossing service boundaries.

7 When to use it (and when not)

Use it when...

- [x] One business action changes data in **several services** with separate databases.
- [x] The business can live with a short "pending" period (eventual consistency).
- [x] The process is long-running: payments, bookings, onboarding, approvals.
- [x] Every step has a sensible undo, or can be placed after the pivot.

Skip it when...

- [ ] All the data is in one database. Use a local transaction.
- [ ] You need strict, immediate consistency (e.g. a ledger that must never show a wrong balance). Keep that data together.
- [ ] Steps can't be compensated and can't be reordered behind a pivot.
- [ ] The "services" are really one tightly coupled thing; maybe merge them first.

Related patterns

Transactional outbox · reliable events

Idempotent consumer · safe duplicates

Event sourcing · state as a log of events

CQRS · separate read models

Process manager / workflow engine

8 Self-check quiz

Q What's the difference between a rollback and a compensation? ✓

A rollback discards uncommitted changes, as if they never happened. A compensation is a new, committed action that semantically reverses an earlier committed one (a refund, a cancellation). Its effects, like a fee or an email, can stay visible.

Q Which step should be the pivot, and why? ✓

The step that is hardest or impossible to undo (often capturing payment). Put compensatable steps before it and steps that are certain to succeed with retries after it, so you never need to undo the pivot.

Q 6 steps with a branch for manual review: choreography or orchestration? ✓

Orchestration. The flow is long and branchy, and the orchestrator keeps it readable, testable and observable in one place.

Q Why do sagas need idempotency? ✓

Messages can be delivered more than once and steps get retried after timeouts. Without idempotency, a step or compensation might run twice (double refund, double reservation).

remember!!

There's no global undo button. Every step needs its own "oops" button, and pressing it twice must be safe.

Further reading

- Hector Garcia-Molina & Kenneth Salem, "Sagas", ACM SIGMOD 1987.
- Chris Richardson, *Microservices Patterns*, chapter 4 "Managing transactions with sagas".
- microservices.io, "Pattern: Saga" and "Pattern: Transactional outbox".
- Microsoft Azure Architecture Center, "Saga distributed transactions pattern".